

Agentenpaket: Repo-Klarstellung & Issue-Liste

Hintergrund

Beim aktuellen Spiel „ZEITRISS“ ist das **Spiel selbst nicht als Programm implementiert**, sondern es wird von einer KI (Spilleitung) aus einem **Datensatz** betrieben.

Das zentrale Konzept: Die KI lädt nur den **Masterprompt** und **19 Wissensmodule (Slots)**, die im `master-index.json` als `slot: true` gekennzeichnet sind.

Alle anderen Dateien im Repository sind **Entwicklungs- oder Testdokumente** (Verzeichnisse `docs/`, `internal/`, `meta/archive/`, `runtime.js`, `tools/` etc.) und dürfen **nicht** in den Wissensspeicher einfließen.

Diese Trennung ist in `AGENTS.md` ausdrücklich festgelegt und muss strikt eingehalten werden:

- `AGENTS.md` richtet sich an Repo-Agenten und gehört **nicht** zum Spielinhalt ¹.
- Es gilt, Spielinhalte (19 Slots + Masterprompt) klar von Dev-Dokumentation zu trennen ².
- **Jede Regeländerung muss in die Wissensdateien gespiegelt werden**; die KI sieht nur diese Dateien ³.
- `runtime.js` ist nur für CI-Tests, nicht für die Spielmechanik ⁴.
- Der Masterprompt definiert das v7-Save-Schema und die Spielinvarianten (Boss-Timing, Szenenanzahl, Attribut-Limits, etc.) ⁵.

Der aktuelle Zustand deutet darauf hin, dass die Trennung nicht konsequent umgesetzt wurde: In der Repo wurden Testskripte, QA-Logdateien und Entwicklungsdokumente nicht klar von den Wissensmodulen abgegrenzt. Dies führte zu Missverständnissen (u. a. bei Save-/Load-Fehlern) und dazu, dass Gameplay-Änderungen nicht in die Wissensdateien zurückgespiegelt wurden. Zudem scheint das Save/Load-System (Level-Up-Gating) nicht vollständig implementiert zu sein.

Identifizierte Probleme

1. **Vermischung von Datensatz und Testcode:** Das Repository enthält zahlreiche Dateien in `docs/`, `internal/qa` und `tools/`, die für Playtests und Entwicklung gedacht sind. Ohne klare Trennung können sich Änderungen oder Bugs aus diesen Quellen in das eigentliche Spiel einschleichen.
2. **Unvollständige Spiegelung von Regeln:** Anpassungen am Save-Schema oder an den Gating-Regeln wurden in QA-Dokumentationen beschrieben, aber nicht konsequent in den Wissensmodulen (Slots) übernommen. Laut `AGENTS.md` dürfen im Spiel nur Regeln gelten, die in den Slots definiert sind ³.
3. **CI-Tests werden als Spielmechanik interpretiert:** Teile in `runtime.js` und automatisierten Tests haben Einzug in Diskussionen über Spielinhalte gehalten, obwohl diese Dateien lediglich CI-Smoke-Tests sind ⁴.
4. **Save/Load-Fehler und Level-Up-Gating:** Nach dem Laden eines Spielstands kann der Spieler erneut Level-Ups vergeben. Laut Spezifikation muss nach jeder Mission die Reihenfolge „Score-Screen → Level-Up (genau eine Wahl) → Save“ eingehalten werden, und das Spiel darf vor der Level-Wahl nicht speichern ⁶. Dieses Anti-Stacking-Gate funktioniert nur, wenn `characters[].level_history` korrekt gepflegt wird – das ist im Moment fehlerhaft.

5. **Fehlende Klarheit bei Rift- und Gruppenmechaniken:** Die Save- und Load-Logik für Core-Missionen, Rift-Operationen und Gruppensplits wurde teilweise in QA-Logs diskutiert, aber noch nicht in den Wissensmodulen verankert.
6. **Commit-Workflow nicht überall befolgt:** `AGENTS.md` gibt ein Pflichtformat für Commit-Messages vor ⁷; Abweichungen erschweren Review und Nachvollziehbarkeit.

Zu erledigende Issues

Damit die Repo den Anforderungen des Spiels entspricht, sollten folgende Issues angelegt und abgearbeitet werden:

1. Repo-Struktur und Klarstellung der Wissensslots

- **Issue:** Dokumentation und Dev-Dateien sind nicht klar von den Wissensmodulen getrennt; manche Tester nehmen an, das gesamte Repository werde geladen.
- **Aufgaben:**
 - Ergänze im `README` einen Abschnitt „Verzeichnisstruktur“ mit klarer Trennung zwischen Wissensmodulen (19 Slots) und Dev/Test-Ordern.
 - Füge einen Hinweis in `master-index.json` ein, der deutlich macht, dass nur Dateien mit `"slot": true` in den Wissensspeicher gelangen.
 - Lege ggf. ein separates Verzeichnis `knowledge/` an, das die 19 Slots enthält, um Verwechslungen zu vermeiden.
- **Referenzen:** Die Rollen der Wissensslots und die Trennung sind in `AGENTS.md` beschrieben ³.

2. Regeländerungen in Wissensdateien spiegeln

- **Issue:** Änderungen an Spielregeln (z. B. Save-Schema v7, Level-Up-Gating, Rift-Mechanik) sind nur in QA-Logs dokumentiert.
- **Aufgaben:**
 - Prüfe alle QA-Logs (`internal/qa/logs`) und `docs/qa` für relevante Regeländerungen (z. B. `ZEITRISS-vereinheitlichungs-fahrplan-2025.md`, `playtest-befund-chargen-save-gate.md`).
 - Übertrage gültige Regeländerungen in die entsprechenden Wissensdateien (`speicher-fortsetzung.md`, `sl-referenz.md`, `core/kampagnenstruktur.md`).
 - Aktualisiere das Save-Schema in `speicher-fortsetzung.md` und `sl-referenz.md`, falls Änderungen notwendig sind.
 - Ergänze testbegleitende Notizen in den QA-Dokumentationen um Verweise auf die geänderten Wissensmodule.
- **Referenzen:** `AGENTS.md` fordert, dass jede Regeländerung in den Wissensdateien verankert werden muss ³.

3. Level-Up-Gating und Save-/Load-Fixes

- **Issue:** Nach einem Laden kann ein Spieler erneut einen Level-Up auswählen. Dies bricht die Anti-Stacking-Regel.
- **Aufgaben:**
 - Untersuche das Save-Schema (`saveGame.v7.schema.json`) und stelle sicher, dass `characters[].level_history` beim Speichern korrekt geschrieben wird und beim Laden vollständig geladen wird.

- Implementiere in den Wissensmodulen eine klare Beschreibung, dass beim Laden geprüft wird: **Für jedes Level gibt es genau einen Eintrag in `level_history`; wenn einer existiert, darf kein weiteres Level-Up gewählt werden.**
- Ergänze in `speicher-fortsetzung.md` und `sl-referenz.md` einen Ablauf für das Level-Up-Gate, inklusive Fehlermeldung ("Kodex: Level-Up ausstehend – Save nach Wahl"), wie in `sl-referenz.md` beschrieben ⁶.
- Teste das Verhalten sowohl für Solo- als auch für Gruppenspiele. Prüfe insbesondere Splits/Merges und Rifts (siehe `core/kampagnenstruktur.md`), dass `level_history` für jeden Charakter konsistent bleibt.
- **Referenzen:** Die Save-Reihenfolge und das Anti-Stacking-Gate sind in `sl-referenz.md` definiert ⁶.

4. Rift- und Gruppenlogik eindeutig definieren

- **Issue:** Die Mechaniken für Kernmissionen, Rift-Operationen und Gruppensplits sind nicht klar in die Wissensmodule integriert; QA-Logs widersprechen sich.
- **Aufgaben:**
- Ergänze in `core/kampagnenstruktur.md` und `sl-referenz.md` eine präzise Beschreibung, wie Rifts während einer laufenden Episode geöffnet werden dürfen und wie Splits/Merges mit dem Save-System interagieren.
- Beschreibe, wie der Loot-Multiplikator und die SG (System-Generator) von der Anzahl der offenen Rifts abhängt und wie diese Information gespeichert wird.
- Prüfe, ob ein Epochen-Lock (Lock auf bestimmte Zeitlinien) beim Laden/Speichern berücksichtigt wird; dokumentiere dies im Wissensmodul.
- **Referenzen:** Hinweise zur Missionsstruktur und Save-Reihenfolge befinden sich in `core/kampagnenstruktur.md` und `sl-referenz.md`.

5. Commit-Workflow und QA-Prozesse durchsetzen

- **Issue:** Commit-Messages und PR-Workflow werden nicht immer nach `AGENTS.md` befolgt; QA-Tests werden ignoriert.
- **Aufgaben:**
- Stelle sicher, dass alle zukünftigen Commits die in `AGENTS.md` definierte Pflicht-Struktur (`type(scope): subject` + klare „Was/Warum/Entscheidungen/Verifikation“-Abschnitte) einhalten ⁷.
- Führe vor jedem Merge `bash scripts/smoke.sh` aus; nur wenn der Smoke-Test grün ist, darf gemerged werden ⁸.
- Erwäge eine GitHub Action, die Commits ohne gültige Struktur blockiert.

Wichtige Referenzdateien

Datei	Zweck	Hinweise
<code>AGENTS.md</code>	Richtlinien für Repo-Agenten; definiert Trennung zwischen Wissensmodulen und Dev-Dokumentation ¹ .	Nicht Teil des Spiels; enthält Commit-Workflow, CI-Hinweise und Invarianten ⁷ .

Datei	Zweck	Hinweise
<code>master-index.json</code>	Liste aller Module mit Attribut <code>slot</code> (true/false). Steuert, welche Dateien in den Wissensspeicher geladen werden.	Prüfen, ob 19 Slots korrekt markiert sind; bei neuen Modulen das Attribut setzen.
<code>speicher-fortsetzung.md</code>	SSOT für das Save/Load-System (v7); beschreibt HQ-Save-Guard, Level-Up-Reihenfolge, Mandatory Fields.	Anpassungen aus QA-Logs müssen hier nachgepflegt werden.
<code>core/sl-referenz.md</code>	Systemreferenz für die KI-Spielleitung; enthält Regeln für Start, Save/Load, Level-Up-Gating und Missionsabläufe ⁶ .	Muss alle gültigen Regeln enthalten; keine Rule of Thumb nur im QA festhalten.
<code>core/kampagnenstruktur.md</code>	Detaillierte Missionsstruktur (Core vs Rift-Ops, Szenen, Arcs); definiert Exfil, Splits/Merges, Loot-Multiplikator.	Erweitern um klare Richtlinien für Rift-Betrieb während laufender Episoden.
<code>docs/qa/...</code>	QA-Befunde und Testberichte.	Dienen als Hinweis auf Bugs und Änderungen, die in die Wissensmodule übernommen werden müssen.
<code>runtime.js</code> / <code>tools/</code>	CI-Smoke-Tests, Skripte für Entwicklung.	Nicht in den Wissensspeicher laden; Änderungen hier müssen in WS gespiegelt werden.

Fazit

Die wichtigste Maßnahme besteht darin, **den Wissensdatensatz sauber von Entwicklungs- und Testmaterial zu trennen**.

Alle Spielregeln müssen in den 19 Wissensmodulen + Masterprompt enthalten sein; Änderungen aus QA-Logs oder Testskripten dürfen nur dann gültig werden, wenn sie dorthin übertragen wurden. Mit den oben beschriebenen Issues können wir sicherstellen, dass das Spiel sauber läuft, der Save-/Load-Prozess in jeder Konstellation funktioniert und die KI-Spielleitung nur konsistente Informationen erhält.

¹ ² ³ ⁴ ⁵ ⁷ ⁸ AGENTS.md

<https://github.com/pchospital-lab/ZEITRISS-md/blob/HEAD/AGENTS.md>

⁶ sl-referenz.md

<https://github.com/pchospital-lab/ZEITRISS-md/blob/main/core/sl-referenz.md>